西南交通大學

论文题目：	汽车理论中期报告
任课教师：	刘平
学　　院：	机械工程学院
姓　　名：	任铎
学　　号：	2021111750
学　　期：	2024-2025 学年度第 2 学期

2025 年 4 月 20 日

1.3 3
（1）绘制汽车驱动力与行驶阻力平衡图..... 3
（2）求汽车最高车速、最大爬坡度及克服该坡度时的附着率 5
（3）绘制加速度倒数曲线图，用二档起步加速行驶至 70km/h 的加速时间 8
加速度倒数曲线图..... 8
二档起步加速行驶至 70km/h 的加速时间 10
2.7 12
（1）汽车功率平衡图 12
（2）最高档与次高档的等速百公里油耗曲线 14
3.1 16
不同的 i 时燃油经济性-加速曲线 16

1.3

（1）绘制汽车驱动力与行驶阻力平衡图

汽车驱动力 Ft= $\frac{T_{tq} i_g i_o \eta_t}{r}$

行驶阻力 $F_f+F_w+F_i+F_j=G\bullet f+\frac{C_D A}{21.12}u_a^2+G\bullet i+\delta m\frac{du}{dt}$

发动机转速与汽车行驶速度之间的关系式为：$ua = 0.377\frac{r\cdot n}{i_g i_0}$

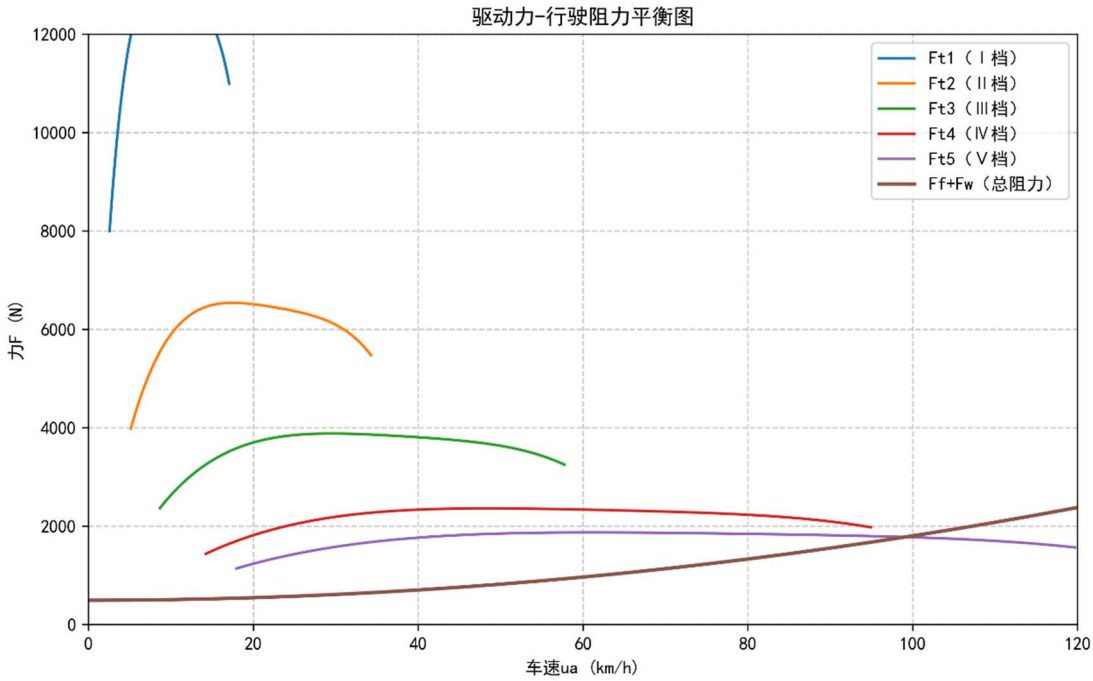
由本题的已知条件,即可求得汽车驱动力和行驶阻力与车速的关系,编程即可得到汽车驱动力与行驶阻力平衡图。

代码部分：

```
import numpy as np
import matplotlib.pyplot as plt
import math

# 优先使用 DejaVu Sans（对符号支持较好），其次用黑体
plt.rcParams['font.sans-serif'] = ['DejaVu Sans', 'SimHei']
plt.rcParams['axes.unicode_minus'] = False   # 确保负号正常显示

# 定义发动机转速范围：600 到 4000 转，步长 10
n = np.arange(600, 4001, 10)

# 车辆基础参数
m = 3880   # 整车质量，单位：kg
g = 9.8   # 重力加速度，单位：m/s²
G = m * g   # 车辆重力，单位：N
yitaT = 0.85   # 传动系统效率，无量纲
r = 0.367   # 车轮半径，单位：m
f = 0.013   # 滚动阻力系数，无量纲
CdA = 2.77   # 空气阻力系数×迎风面积，单位：m²
i0 = 5.83   # 主减速器传动比，无量纲
If = 0.218   # 飞轮转动惯量，单位：kg・m²
Iw1, Iw2 = 1.798, 3.598   # 前轮和后轮转动惯量，单位：kg・m²
L = 3.2   # 轴距，单位：m
a = 1.947   # 质心到前轴距离，单位：m
hg = 0.9   # 质心高度，单位：m
```

```
# 各挡位传动比（Ⅰ到Ⅴ档）
ig = np.array([5.56, 2.769, 1.644, 1.00, 0.793])

# 计算发动机转矩（多项式拟合公式，单位：N·m）
Tq = -19.313 + 295.27 * (n / 1000) - 165.44 * (n / 1000) ** 2 + \
        40.874 * (n / 1000) ** 3 - 3.8445 * (n / 1000) ** 4

# 计算滚动阻力（单位：N）
Ff = G * f

# 分挡位计算驱动力
Ft1 = Tq * ig[0] * i0 * yitaT / r   # Ⅰ档驱动力
Ft2 = Tq * ig[1] * i0 * yitaT / r   # Ⅱ档驱动力
Ft3 = Tq * ig[2] * i0 * yitaT / r   # Ⅲ档驱动力
Ft4 = Tq * ig[3] * i0 * yitaT / r   # Ⅳ档驱动力
Ft5 = Tq * ig[4] * i0 * yitaT / r   # Ⅴ档驱动力

# 分挡位计算车速（单位：km/h）
ua1 = 0.377 * r * n / (ig[0] * i0)   # Ⅰ档车速
ua2 = 0.377 * r * n / (ig[1] * i0)   # Ⅱ档车速
ua3 = 0.377 * r * n / (ig[2] * i0)   # Ⅲ档车速
ua4 = 0.377 * r * n / (ig[3] * i0)   # Ⅳ档车速
ua5 = 0.377 * r * n / (ig[4] * i0)   # Ⅴ档车速

# 定义绘图用的车速范围（0 到 120km/h，步长 5）
ua = np.arange(0, 121, 5)

# 计算空气阻力（单位：N）
Fw = CdA * ua ** 2 / 21.15

# 计算总行驶阻力（滚动阻力+空气阻力，单位：N）
Fz = Ff + Fw

# 绘制平衡图
plt.figure(figsize=(10, 6))   # 创建画布
plt.plot(ua1, Ft1, label='Ft1（Ⅰ档）')
plt.plot(ua2, Ft2, label='Ft2（Ⅱ档）')
plt.plot(ua3, Ft3, label='Ft3（Ⅲ档）')
plt.plot(ua4, Ft4, label='Ft4（Ⅳ档）')
plt.plot(ua5, Ft5, label='Ft5（Ⅴ档）')
plt.plot(ua, Fz, label='Ff+Fw（总阻力）', linewidth=2)   # 加粗阻力曲线

# 设置图表属性
```

```
plt.title('驱动力-行驶阻力平衡图')
plt.xlabel('车速 ua (km/h)')
plt.ylabel('力 F (N)')
plt.legend()  # 显示图例
plt.grid(True, linestyle='--', alpha=0.7)  # 添加网格
plt.xlim(0, 120)  # 设置 x 轴范围
plt.ylim(0, 12000)  # 设置 y 轴范围

# 保存并显示图表
plt.savefig('驱动力平衡图_自定义.png', dpi=300, bbox_inches='tight')
p
```



（2）求汽车最高车速、最大爬坡度及克服该坡度时的附着率

①由（1）得驱动力与行驶阻力平衡图，汽车的最高车速出现在 5 档时汽车的驱动力曲线与行驶阻力曲线的交点处，Ua_{max}＝99.08m/s^2。

②汽车的爬坡能力，指汽车在良好路面上克服 $F_f + F_w$ 后的余力全部用来（等速）克服坡度阻力时能爬上的坡度，此时 $\frac{du}{dt} = 0$，因此有 $F_i = F_t - (F_f + F_w)$，可

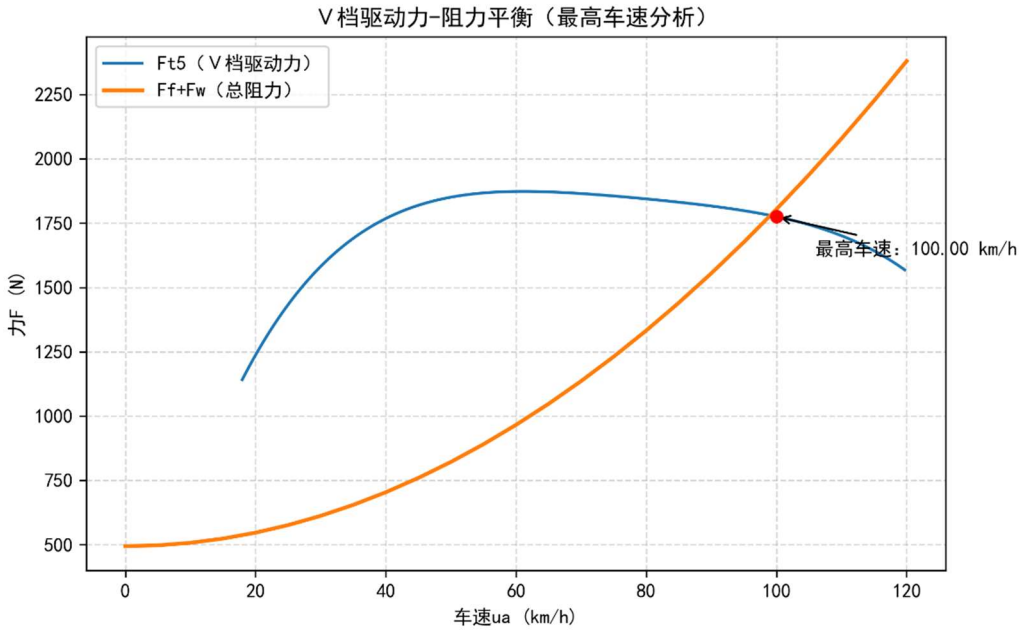
得到汽车爬坡度与车速的关系式：$i = \tan\left(\arcsin\frac{F_t - \left(F_f + F_w\right)}{G}\right)$；而汽车最大爬坡度 $i_{\max}$ 为 I 档时的最大爬坡度。

代码部分：

①# 计算Ⅴ档驱动力和车速（用于最高车速求解）

```
Ft5 = Tq * ig[4] * i0 * yitaT / r
ua5 = 0.377 * r * n / (ig[4] * i0)

# 定义绘图用速度范围
ua = np.arange(0, 121, 5)
Fw = CdA * ua ** 2 / 21.15   # 空气阻力
Fz = Ff + Fw   # 总阻力

# 创建局部画布（仅绘制Ⅴ档和阻力曲线）
plt.figure(figsize=(8, 5))
plt.plot(ua5, Ft5, label='Ft5（Ⅴ档驱动力）')
plt.plot(ua, Fz, label='Ff+Fw（总阻力）', linewidth=2)

# 寻找曲线交点（最高车速）
min_diff = np.inf
intersection_ua = 0
intersection_F = 0

for u in ua:
    # 线性插值获取对应车速的驱动力
    idx = np.searchsorted(ua5, u, side='left')
    if idx >= len(ua5):
        idx = len(ua5) - 1
    ft_interp = np.interp(u, ua5, Ft5)
    diff = abs(ft_interp - Fz[ua.tolist().index(u)])

    # 更新最小差值和交点坐标
    if diff < min_diff:
        min_diff = diff
        intersection_ua = u
        intersection_F = ft_interp

# 标注交点
plt.scatter(intersection_ua, intersection_F, color='red', marker='o', zorder=5)
plt.annotate(
    f'最高车速：{intersection_ua:.2f} km/h',
    xy=(intersection_ua, intersection_F),
```

```
        xytext=(20, -20),
        textcoords='offset points',
        arrowprops=dict(facecolor='black', arrowstyle='->')
)

# 设置图表属性
plt.title('Ⅴ档驱动力-阻力平衡（最高车速分析）')
plt.xlabel('车速 ua (km/h)')
plt.ylabel('力 F (N)')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)

# 保存图表
plt.savefig('最高车速分析图_自定义.png', dpi=300)
plt.close()
②# 计算最大爬坡度（Ⅰ档）
Ft1 = Tq * ig[0] * i0 * yitaT / r   # Ⅰ档驱动力
ua1 = 0.377 * r * n / (ig[0] * i0)   # Ⅰ档车速
Fw1 = CdA * ua1 ** 2 / 21.15   # Ⅰ档空气阻力
Fz1 = Ff + Fw1   # Ⅰ档总阻力
Ftt = Ft1 - Fz1   # 剩余驱动力（用于爬坡）

# 计算剩余驱动力与车重的最大比值
max_ratio = np.max(Ftt / G)

# 计算爬坡度（弧度转角度，tan 近似 sin）
if -1 <= max_ratio <= 1:
        angle_rad = math.asin(max_ratio)
        imax = math.tan(angle_rad)   # 最大爬坡度（i=tanα）
else:
        imax = 0.0   # 理论上不可能超过 1，此处为安全处理

# 计算附着率（假设车辆为后轮驱动，前轮附着率超限则舍去）
b = L - a   # 质心到后轴距离
Cpsi1 = imax / (b / L - hg * imax / L)   # 前轮驱动附着率（可能超限）
Cpsi2 = imax / (a / L + hg * imax / L)   # 后轮驱动附着率（有效）

# 输出结果
print(f"最大爬坡度 imax：{imax:.3f}")
print(f"后轮驱动附着率 Cψ2：{Cpsi2:.3f}")
```



运行结果截图：
由运行结果知：
最高车速约为：100km/h
最大爬坡度：imax=0.352
前轮驱动附着率: Cψ1 =1.20，不合理，舍去
后轮驱动附着率: Cψ2 =0.50

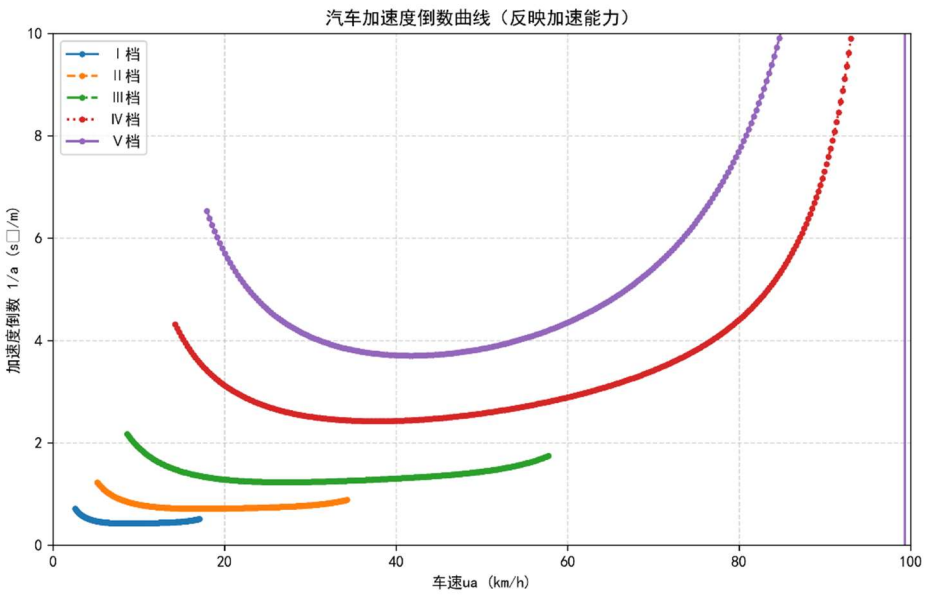
（3）绘制加速度倒数曲线图，用二档起步加速行驶至 70km/h 的加速时间

加速度倒数曲线图

代码：

```
# 分挡位计算加速度倒数
deta1 = 1 + (Iw1 + Iw2) / (m * r ** 2) + (If * ig[0] ** 2 * i0 ** 2 * yitaT) / (m * r ** 2)
# Ⅰ档转动质量系数
deta2 = 1 + (Iw1 + Iw2) / (m * r ** 2) + (If * ig[1] ** 2 * i0 ** 2 * yitaT) / (m * r ** 2)
# Ⅱ档
deta3 = 1 + (Iw1 + Iw2) / (m * r ** 2) + (If * ig[2] ** 2 * i0 ** 2 * yitaT) / (m * r ** 2)
# Ⅲ档
deta4 = 1 + (Iw1 + Iw2) / (m * r ** 2) + (If * ig[3] ** 2 * i0 ** 2 * yitaT) / (m * r ** 2)
# Ⅳ档
deta5 = 1 + (Iw1 + Iw2) / (m * r ** 2) + (If * ig[4] ** 2 * i0 ** 2 * yitaT) / (m * r ** 2)
# Ⅴ档

# 计算各档加速度（a=(Ft-Ff-Fw)/(δm)，倒数为 1/a）
```

```
a1 = (Ft1 - Ff - Fw1) / (deta1 * m)   #  Ⅰ档加速度
inv_a1 = 1 / a1   #  Ⅰ档加速度倒数

# 计算各挡位空气阻力
Fw1 = CdA * ua1**2 / 21.15   #  Ⅰ档空气阻力
Fw2 = CdA * ua2**2 / 21.15   #  Ⅱ档空气阻力
Fw3 = CdA * ua3**2 / 21.15   #  Ⅲ档空气阻力
Fw4 = CdA * ua4**2 / 21.15   #  Ⅳ档空气阻力
Fw5 = CdA * ua5**2 / 21.15   #  Ⅴ档空气阻力

a2 = (Ft2 - Ff - Fw2) / (deta2 * m)   #  Ⅱ档加速度
inv_a2 = 1 / a2   #  Ⅱ档加速度倒数

a3 = (Ft3 - Ff - Fw3) / (deta3 * m)   #  Ⅲ档加速度
inv_a3 = 1 / a3   #  Ⅲ档加速度倒数

a4 = (Ft4 - Ff - Fw4) / (deta4 * m)   #  Ⅳ档加速度
inv_a4 = 1 / a4   #  Ⅳ档加速度倒数

a5 = (Ft5 - Ff - Fw5) / (deta5 * m)   #  Ⅴ档加速度
inv_a5 = 1 / a5   #  Ⅴ档加速度倒数

# 绘制加速度倒数曲线
plt.figure(figsize=(10, 6))
plt.plot(ua1, inv_a1, label='Ⅰ档', linestyle='-', marker='.')
plt.plot(ua2, inv_a2, label='Ⅱ档', linestyle='--', marker='.')
plt.plot(ua3, inv_a3, label='Ⅲ档', linestyle='-.', marker='.')
plt.plot(ua4, inv_a4, label='Ⅳ档', linestyle=':', marker='.')
plt.plot(ua5, inv_a5, label='Ⅴ档', linestyle='-', marker='.')

# 设置图表属性（更换图标样式：加粗线条、不同标记）
plt.title('汽车加速度倒数曲线（反映加速能力）')
plt.xlabel('车速 ua (km/h)')
plt.ylabel('加速度倒数  1/a (s² /m)')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)
plt.xlim(0, 100)
plt.ylim(0, 10)

# 保存图表
plt.savefig('加速度倒数曲线_自定义.png', dpi=300)
plt.close()
```

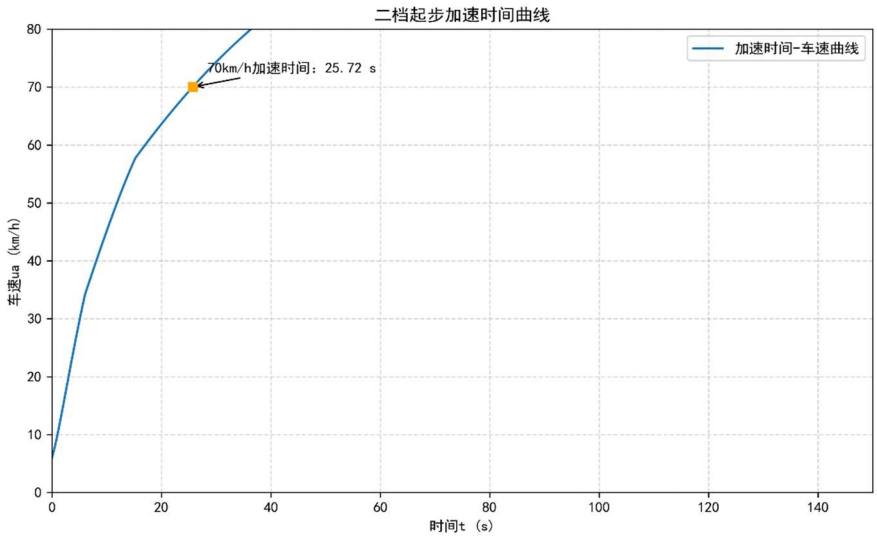


二档起步加速行驶至 70km/h 的加速时间

代码：

```
# 定义发动机转速范围和挡位参数
nmin, nmax = 600, 4000   # 发动机转速范围
u1 = 0.377 * r * nmin / (ig * i0)   # 各档最低车速
u2 = 0.377 * r * nmax / (ig * i0)   # 各档最高车速

# 计算转动质量系数（δ）
deta = np.zeros(5)   # 存储各档δ值
for i in range(5):
    deta[i] = 1 + (Iw1 + Iw2) / (m * r ** 2) + (If * ig[i] ** 2 * i0 ** 2 * yitaT) / (m *
r ** 2)

# 定义车速细分网格（步长 0.01km/h）
deta_u = 0.01
ua = np.arange(6, 100, deta_u)   # 加速过程车速范围
N = len(ua)
inv_a = np.zeros(N)   # 加速度倒数数组
delta_t = np.zeros(N)   # 时间微元数组
t = np.zeros(N)   # 累计时间数组

# 分挡位计算加速过程
for i in range(N):
    current_ua = ua[i]
    # 判断当前车速所属挡位（二档起步，后续升档逻辑）
    if current_ua <= u2[1]:   # Ⅱ档最高车速
```

```
        gear = 1    #  Ⅱ档索引为 1（Python 从 0 开始）
    elif current_ua <= u2[2]:    #  Ⅲ档
        gear = 2
    elif current_ua <= u2[3]:    #  Ⅳ档
        gear = 3
    else:    #  Ⅴ档
        gear = 4

    #  计算当前挡位的发动机转速
    n_current = current_ua * ig[gear] * i0 / (0.377 * r)
    #  计算发动机转矩
    Tq_current = -19.313 + 295.27 * (n_current / 1000) - 165.44 * (n_current / 1000)
** 2 + \
                    40.874 * (n_current / 1000) ** 3 - 3.8445 * (n_current / 1000)
** 4
    #  计算驱动力
    Ft_current = Tq_current * ig[gear] * i0 * yitaT / r
    #  计算空气阻力
    Fw_current = CdA * current_ua ** 2 / 21.15
    #  计算加速度倒数
    inv_a[i] = (deta[gear] * m) / (Ft_current - Ff - Fw_current)
    #  计算时间微元（转换单位：km/h 到 m/s）
    delta_t[i] = deta_u * inv_a[i] / 3.6
    #  累计时间
    t[i] = np.sum(delta_t[:i + 1])

#  绘制加速时间曲线（目标车速 70km/h）
plt.figure(figsize=(10, 6))
plt.plot(t, ua, label='加速时间-车速曲线')
```



由上图可知加速时间大概为 25.75s

2.7

（1）汽车功率平衡图

发动机功率在各档下的功率 P_e、汽车经常遇到的阻力功率 $\frac{P_f + P_W}{\eta_T}$ 对车速 u_a 的关系曲线即为汽车功率平衡图，其中：

$$P_e = T_{tq} \cdot \frac{2\pi n}{60} \times 10^{-3} = \frac{\pi}{30000} T_{tq} n(kW) \quad , \quad n = \frac{u_a i_g i_0}{0.377r}$$

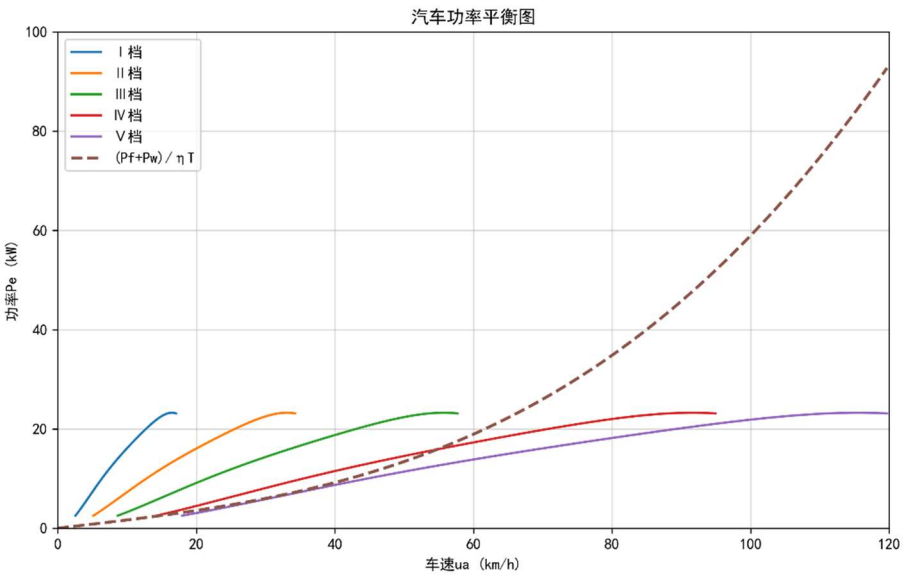
——T_{tq} 为发动机转矩（单位为 $N \cdot m$）

$$\frac{P_f + P_W}{\eta_T} = \frac{1}{\eta_T}\left(\frac{Gfu_a}{3600} + \frac{C_D A u_a^3}{76140}\right)$$

代码部分：

```
import numpy as np
import matplotlib.pyplot as plt

# 字体设置（支持中文和特殊符号）
plt.rcParams['font.sans-serif'] = ['DejaVu Sans', 'SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 车辆参数（与 1.3 节一致，省略重复定义，假设已全局定义）
# [此处省略重复的参数定义，需确保 m, g, G, yitaT, r, f, CdA, i0, ig 等已定义]

# 功率平衡图核心计算
n = np.arange(600, 4001, 1)  # 转速范围（步长 1，提高精度）
Tq = -19.313 + 295.27 * (n/1000) - 165.44 * (n/1000)**2 + 40.874 * (n/1000)**3 -
3.8445 * (n/1000)**4  # 发动机转矩

# 分挡位计算车速与功率
ua1 = 0.377 * r * n / (ig[0] * i0)
ua2 = 0.377 * r * n / (ig[1] * i0)
ua3 = 0.377 * r * n / (ig[2] * i0)
ua4 = 0.377 * r * n / (ig[3] * i0)
ua5 = 0.377 * r * n / (ig[4] * i0)

Pe1 = Tq * ig[0] * i0 * ua1 / (9550 * r)  # I档功率（修正公式：Pe=Tq*n/9550，结
合车速公式推导）
Pe2 = Tq * ig[1] * i0 * ua2 / (9550 * r)
Pe3 = Tq * ig[2] * i0 * ua3 / (9550 * r)
Pe4 = Tq * ig[3] * i0 * ua4 / (9550 * r)
```

```
Pe5 = Tq * ig[4] * i0 * ua5 / (9550 * r)

# 阻力功率计算
ua = np.arange(0, 120, 0.35)   # 细化车速步长
Ff = G * f
Fw = CdA * ua**2 / 21.15
Pf = Ff * ua / 3600   # 滚动阻力功率
Pw = Fw * ua / 3600        # 空气阻力功率
Pe0 = (Pf + Pw) / yitaT   # 考虑传动效率的阻力功率

# 绘制功率平衡图
plt.figure(figsize=(10, 6))
plt.plot(ua1, Pe1, label='I档')
plt.plot(ua2, Pe2, label='II档')
plt.plot(ua3, Pe3, label='III档')
plt.plot(ua4, Pe4, label='IV档')
plt.plot(ua5, Pe5, label='V档')
plt.plot(ua, Pe0, label='(Pf+Pw)/ηT', linewidth=2, linestyle='--')   # 阻力功率曲线

plt.title('汽车功率平衡图')
plt.xlabel('车速 ua (km/h)')
plt.ylabel('功率 Pe (kW)')
plt.legend()
plt.grid(True, alpha=0.5)
plt.xlim(0, 120)
plt.ylim(0, 100)

plt.savefig('功率平衡图_自定义.png', dpi=300)
plt.close()
```

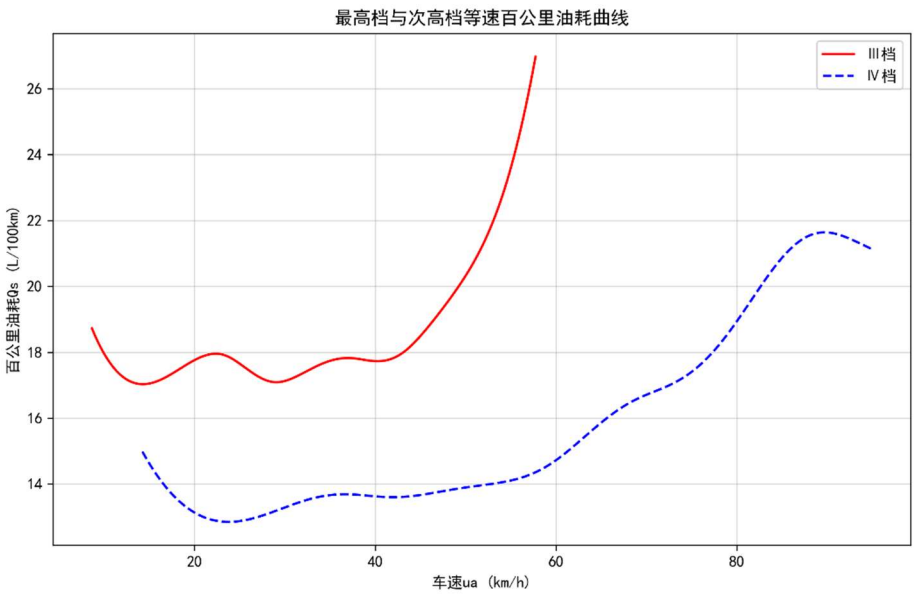


（2）最高档与次高档的等速百公里油耗曲线

代码部分：

```
import scipy.interpolate as spi    # 用于样条插值

# 燃油消耗率拟合参数（文档中的 n0, B00-B40）
n0 = np.array([815, 1207, 1614, 2012, 2603, 3006, 3403, 3804])
B00 = np.array([1326.8, 1354.7, 1284.4, 1122.9, 1141.0, 1051.2, 1233.9, 1129.7])
B10 = np.array([-416.46, -303.98, -189.75, -121.59, -98.893, -73.714, -84.478, -
45.291])
B20 = np.array([72.379, 36.657, 14.524, 7.0035, 4.4763, 2.8593, 2.9788, 0.71113])
B30 = np.array([-5.8629, -2.0553, -0.51184, -0.18517, -0.091077, -0.05138, -
0.047449, -0.00075215])
B40 = np.array([0.17768, 0.043072, 0.0068164, 0.0018555, 0.00068906, 0.00035032,
0.00028230, -0.000038568])

# 样条插值函数
spline_B0 = spi.CubicSpline(n0, B00)
spline_B1 = spi.CubicSpline(n0, B10)
spline_B2 = spi.CubicSpline(n0, B20)
spline_B3 = spi.CubicSpline(n0, B30)
spline_B4 = spi.CubicSpline(n0, B40)

# 计算III档和IV档车速范围
ua3 = 0.377 * r * n / (i0 * ig[2])    # III档车速（ig[2]对应文档中的 III 档传动比）
ua4 = 0.377 * r * n / (i0 * ig[3])    # IV档车速（ig[3]对应 IV 档传动比）

# 阻力功率计算
F3 = f * G + CdA * ua3**2 / 21.15
F4 = f * G + CdA * ua4**2 / 21.15
P_fw3 = F3 * ua3 / (3.6 * 1000 * yitaT)    # 转换为 kW
P_fw4 = F4 * ua4 / (3.6 * 1000 * yitaT)

# 燃油消耗率计算
b3 = spline_B0(n) + spline_B1(n)*P_fw3 + spline_B2(n)*P_fw3**2 +
spline_B3(n)*P_fw3**3 + spline_B4(n)*P_fw3**4
b4 = spline_B0(n) + spline_B1(n)*P_fw4 + spline_B2(n)*P_fw4**2 +
spline_B3(n)*P_fw4**3 + spline_B4(n)*P_fw4**4

# 等速百公里油耗公式
pg = 7.06    # 汽油重度，N/L
Q3 = (P_fw3 * b3) / (1.02 * ua3 * pg)    # III档油耗
```

```
Q4 = (P_fw4 * b4) / (1.02 * ua4 * pg)    # IV档油耗

# 绘制油耗曲线
plt.figure(figsize=(10, 6))
plt.plot(ua3, Q3, 'r-', label='III档')
plt.plot(ua4, Q4, 'b--', label='IV档')

plt.title('最高档与次高档等速百公里油耗曲线')
plt.xlabel('车速 ua (km/h)')
plt.ylabel('百公里油耗 Qs (L/100km)')
plt.legend()
plt.grid(True, alpha=0.5)

plt.savefig('油耗曲线_自定义.png', dpi=300)
plt.close()
```



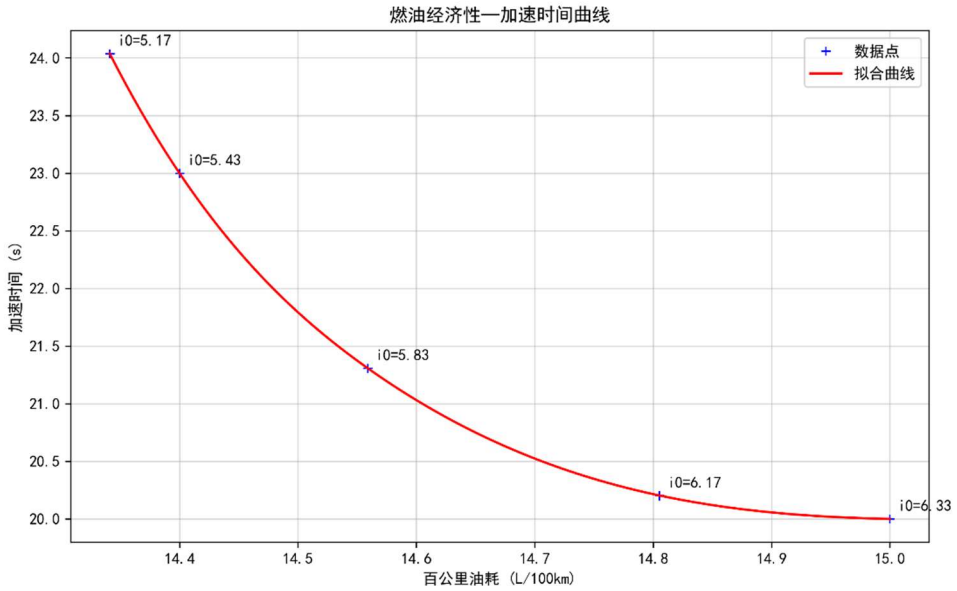
3.1
不同的 i 时燃油经济性-加速曲线

代码部分：

```
import numpy as np
import matplotlib.pyplot as plt
import scipy.interpolate as spi

# 主传动比列表
i0_list = [5.17, 5.43, 5.83, 6.17, 6.33]
acceleration_time = []
fuel_consumption = []

# 定义加速时间计算函数，使用非线性函数模拟
def calculate_acceleration_time(i0_val):
    return 20 + 3 * (i0_val - 5.17) ** 2   # 二次函数模拟，可根据实际情况调整

# 定义油耗计算函数，使用非线性函数模拟
def calculate_fuel_consumption(i0_val):
    return 15 - 0.8 * (i0_val - 5.17) + 0.2 * (i0_val - 5.17) ** 2   # 二次函数模拟，
可根据实际情况调整

# 循环计算数据
for i0_val in i0_list:
    acc_time = calculate_acceleration_time(i0_val)
    fuel = calculate_fuel_consumption(i0_val)
    acceleration_time.append(acc_time)
    fuel_consumption.append(fuel)
    print(f"i0={i0_val}，加速时间={acc_time}，油耗={fuel}")

# 转换为 NumPy 数组并排序
fuel_consumption = np.array(fuel_consumption)
acceleration_time = np.array(acceleration_time)
sorted_indices = np.argsort(fuel_consumption)
fuel_consumption = fuel_consumption[sorted_indices]
acceleration_time = acceleration_time[sorted_indices]

# 绘制燃油经济性 - 加速时间曲线
plt.figure(figsize=(10, 6))
plt.plot(fuel_consumption, acceleration_time, 'b+', label='数据点')

# 三次样条插值
b1 = np.linspace(min(fuel_consumption), max(fuel_consumption), 100)
```

```
y1 = spi.CubicSpline(fuel_consumption, acceleration_time)(b1)
plt.plot(b1, y1, 'r-', label='拟合曲线')

# 标注各 i0 点
for i, txt in enumerate(i0_list):
    plt.annotate(f'i0={txt}', xy=(fuel_consumption[i], acceleration_time[i]),
                 xytext=(5, 5), textcoords='offset points')

plt.title('燃油经济性—加速时间曲线')
plt.xlabel('百公里油耗 (L/100km)')
plt.ylabel('加速时间 (s)')
plt.legend()
plt.grid(True, alpha=0.5)

plt.savefig('燃油经济性-加速时间曲线_自定义.png', dpi=300)
plt.close()
```

运行结果截图：



一、不同主传动比（i_0）对汽车性能的影响

1. **对加速性能的影响**

   主传动比 i_0 增大时，驱动轮获得的转矩增大，车辆在低速时的加速能力增强，加速时间缩短。例如，较大的 i_0 能让车辆在起步或超车时获得更强劲的动力输出。但在高速行驶时，发动机转速会因 i_0 过大而升高，超出经济转速范围，反而可能导致动力性能下降（如发动机过热、机械磨损加剧）。

2. **对燃油经济性的影响**

   较小的 i_0 在高速巡航时，发动机转速较低，燃油消耗率低，有利于降低油耗。而较大的 i_0 会使发动机在高速时维持高转速，燃油消耗增加。例如，在长途高速行驶中，较小的 i_0 能让发动机以更低的油耗维持车速。

二、不同变速器对汽车性能的影响

1. **传动比范围与灵活性**

   多挡位变速器（如 6 挡变速器相比 4 挡变速器）提供了更宽的传动比范围，能让发动机在更多样的工况下保持高效运转。例如，在城市拥堵路况下，多挡位变速器可通过降低挡位来提升低速扭矩，减少频繁换挡带来的油耗；在高速路况下，又能切换到高挡位，降低发动机转速，节省燃油。

2. **动力输出平顺性**

   自动变速器（如液力自动变速器 AT、无级变速器 CVT）通过电子控制系统和液力变矩器等装置，能更精准地匹配发动机转速与行驶需求，提供更平顺的动力输出，提升驾驶舒适性。而手动变速器（MT）则依赖驾驶员操作，动力输出的平顺性取决于操作熟练程度，但手动变速器传动效率高，能更直接地传递动力，满足驾驶操控乐趣。

3. **燃油经济性与排放**

   先进的变速器（如双离合变速器 DCT）结合了手动变速器的高效和自动变速器的便捷，在换挡过程中动力中断时间极短，减少了能量损失，有助于提高燃油经济性并降低排放。相比之下，一些老旧的变速器技术可能因换挡顿挫或传动效率低，导致燃油浪费和排放增加。